TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA ĐIỆN TỬ VIỄN THÔNG

Thân Thế Trí Dũng
Mã sinh viên: 23020730

BÁO CÁO GIỮ KỲ

Môn học: Môn học: Lập trình Robot với Ros

HÀ NỘI – 2026

Mục lục

1 Giới thiệu đề tài **3**

1.1 Phương án điều khiển 3

1.2 Tay máy 4

1.3 Hệ thống công tác và Cảm biến 4

2 Quá trình thiết kế và Lập trình **6**

2.1 Thiết kế mô hình 3D và Chuẩn hóa URDF 6

2.1.1 Xử lý điểm gốc tọa độ (Root Link) 6

2.2 Nhúng hệ thống điều khiển và Cảm biến (Gazebo Plugins) 7

2.2.1 Hệ thống truyền động bánh xích 7

2.2.2 Hệ thống điều khiển Tay máy (ROS 2 Control) 8

2.2.3 Tích hợp Cảm biến LiDAR và IMU 9

2.2.4 Cấu trúc tổng thể của Package 11

2.2.5 Cấu trúc tổng thể của package 12

2.2.6 Cấu trúc cây động học (Kinematic Tree) của file URDF . . . 13

2.2.7 Giải phẫu cấu trúc XML của tệp URDF trích xuất 16

3 Kết quả thu được **19**

3.1 Khởi chạy chương trình 19

3.2 Hiển thị trong mô phỏng 19

4 Kết luận **22**

4.1 Đã thực hiện 22

4.2 Công việc cần hoàn thiện 22

4.3 Lỗi liên quan đến việc xuất URDF 24

4.3.1 Lỗi quan hệ cấu trúc cây - tay máy 24

4.3.2 Lỗi quan hệ cấu trúc cây - xích 27

4.4 Lỗi liên quan đến mô phỏng trên gazebo 29
4.4.1 Lỗi nổ vật lý của agv cũng như không hiên agv khi nhét vô mô phỏng vật lý gazebo 29
4.4.2 Lỗi lidar không nhận diện được vật thể 32
4.4.3 Lỗi lidar không nhận diện được vật thể 34
4.5 Thành phẩm sau cùng cũng như link github sản phẩm 34

Chương 1

Giới thiệu đề tài

Trong bối cảnh tự động hóa công nghiệp và logistics đang phát triển mạnh mẽ, Robot tự hành (AGV - Automated Guided Vehicle) đóng vai trò then chốt trong việc vận chuyển hàng hóa linh hoạt. Đề tài giữa kỳ được lựa chọn là thiết kế, mô phỏng và điều khiển một hệ thống AGV sử dụng cơ cấu di chuyển bằng bánh xích, kết hợp với tay máy công nghiệp 2 bậc tự do (2-DOF) trên nền tảng hệ điều hành Robot (ROS 2 Humble).

1.1 Phương án điều khiển

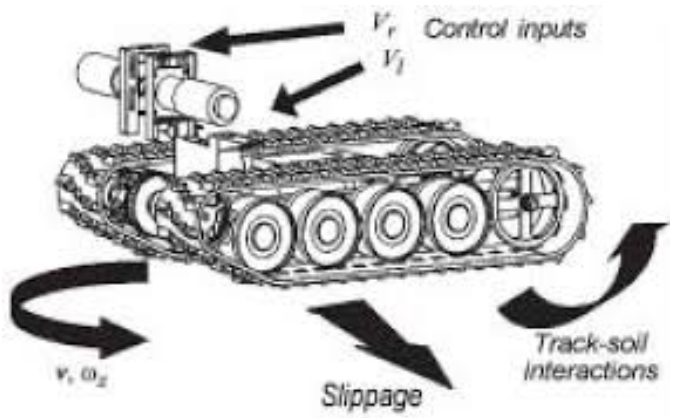
Với yêu cầu robot di chuyển bằng bánh xích, sau khi tham khảo các plugin khác nhau, em quyế định lựa chọn phương thức skid-steering cho mô phỏng robot bánh xích vì các lí do sau: **Skid-Steering** (lái trượt) thay vì Differential Drive (vi sai) thông thường.

- **Bản chất vật lý:** Không giống như xe ô tô có hệ thống lái (Ackermann) chuyển hướng bằng cách bẻ góc bánh xe, xe bánh xích chuyển hướng bằng cách tạo ra sự chênh lệch vận tốc giữa dải xích bên trái và bên phải. Khi đó, sự trượt (slippage) giữa xích và mặt đất sẽ sinh ra mô-men xoay thân xe.

- **Mô hình toán học:** Gọi v là vận tốc tịnh tiến, ω là vận tốc góc của xe. Khi đó, vận tốc của dải xích hai bên (v_R, v_L) được tính toán thông qua bán kính bánh dẫn động r và khoảng cách giữa hai dải xích B:

$$v = \frac{r(\omega_R + \omega_L)}{2} \quad ; \quad \omega = \frac{r(\omega_R - \omega_L)}{B}$$

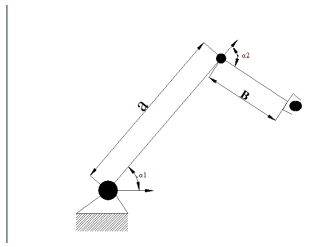
- **Ưu điểm trong mô phỏng:** Mô hình Skid-Steering của Gazebo xử lý rất tốt các ma sát trượt và tự động tính toán tâm quay tức thời (ICR - Instantaneous Center of Rotation), giúp robot bám đường tốt và có khả năng quay tại chỗ (Zero-radius turn) ổn định.



Hình 1.1: Minh hoạ động học Skid-Steering.

1.2 Tay máy

Hệ thống được trang bị một tay máy cơ khí với cấu trúc hai bậc tự do (2-DOF), hỗ trợ robot thực hiện các thao tác tương tác vật lý và xử lý vật thể linh hoạt trong không gian mô phỏng



Hình 1.2: Mô hình cánh tay robot 2 khớp xoay.

1.3 Hệ thống công tác và Cảm biến

- **Tay máy 2-DOF:** Gồm 2 khớp quay (Revolute joint) đóng vai trò như khớp vai và khớp khuỷu, kết hợp với một nòng súng/tay gắp (End-effector) ở đầu

cuối. Tay máy cho phép AGV tương tác vật lý với môi trường xung quanh.

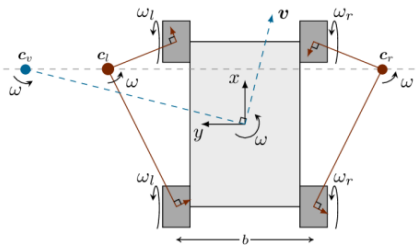
- **LiDAR (Light Detection and Ranging):** Cảm biến quét tia laser 360 độ. Đây là cảm biến sống còn để thực hiện thuật toán SLAM (Xây dựng bản đồ) và Navigation (Tránh vật cản động) trong không gian 2D.
- **IMU (Inertial Measurement Unit):** Cảm biến đo lường quán tính (gồm con quay hồi chuyển và gia tốc kế). Nhiệm vụ của IMU là cung cấp thông tin về độ nghiêng (Roll, Pitch), hướng la bàn (Yaw) và gia tốc tuyến tính để tính toán thăng bằng cho robot, đặc biệt khi đi qua địa hình gồ ghề.
- **Encoder:** Bộ mã hóa vòng quay được giả lập trực tiếp trên các khớp bánh xe thông qua `joint_state_broadcaster`, giúp cung cấp dữ liệu Odometry (đo lường quãng đường) để robot biết mình đã di chuyển được bao xa.

Chương 2

Quá trình thiết kế và Lập trình

2.1 Thiết kế mô hình 3D và Chuẩn hóa URDF

Robot được thiết kế nguyên bản trên phần mềm CAD (SolidWorks), sau đó trích xuất các thành phần cơ khí (Links) dưới dạng lưới `.STL`. Quá trình định nghĩa khung xương động học được viết bằng ngôn ngữ XML thông qua chuẩn **URDF** (Unified Robot Description Format) bằng Plugins SolidWorks to URDF Exporter.



Hình 2.1: Mô hình agv trong solid.

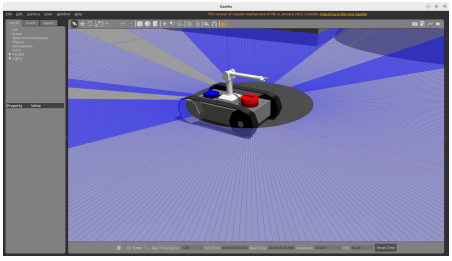
2.1.1 Xử lý điểm gốc tọa độ (Root Link)

Một vấn đề kinh điển trong Gazebo là nếu liên kết gốc (`base_link`) được gán khối lượng và quán tính, hệ thống mô phỏng vật lý có thể bị văng (Crash). Để khắc phục, em đã thiết lập một liên kết ảo `dummy_root_link` không có thuộc tính vật lý, nối cố định với thân xe thực sự.

Listing 2.1: Khởi tạo liên kết gốc ảo chống lỗi vật lý cho Gazebo

```
<link name="dummy_root_link" />
```

```
<joint name="dummy_joint" type="fixed">
  <parent link="dummy_root_link" />
  <child link="base_link" />
  <origin xyz="0 0 0.06" rpy="0 0 0" />
</joint>
```



Hình 2.2: Mô hình được mô phỏng bởi Gazebo.

2.2 Nhúng hệ thống điều khiển và Cảm biến (Gazebo Plugins)

2.2.1 Hệ thống truyền động bánh xích

Do xích là một cơ cấu mảng phức tạp, việc mô phỏng chi tiết từng mắt xích sẽ gây quá tải tài nguyên tính toán. Giải pháp tối ưu là sử dụng bánh xe ảo ở 4 góc và dùng plugin `libgazebo_ros_diff_drive.so` cấu hình dạng 4 bánh (Skid Steer) để bao bọc chuyển động.

Listing 2.2: Cấu hình Plugin truyền động Skid-Steering

```
<gazebo>
  <plugin name='skid_steer_drive' filename='
     libgazebo_ros_diff_drive.so'>
    <ros>
      <remapping>cmd_vel:=cmd_vel</remapping>
      <remapping>odom:=odom</remapping>
    </ros>
    <update_rate>50</update_rate>
    <num_wheel_pairs>2</num_wheel_pairs>
    <left_joint>lw1</left_joint>
```

```
    <right_joint>rw1</right_joint>
    <left_joint>lw2</left_joint>
    <right_joint>rw2</right_joint>
    <wheel_separation>0.12</wheel_separation>
    <wheel_diameter>0.066</wheel_diameter>
    <max_wheel_torque>20</max_wheel_torque>
    <publish_odom>true</publish_odom>
    <odometry_frame>odom</odometry_frame>
    <robot_base_frame>dummy_root_link</robot_base_frame>
  </plugin>
</gazebo>
```

2.2.2 Hệ thống điều khiển Tay máy (ROS 2 Control)

Khác với ROS 1 điều khiển trực tiếp qua plugin, hệ thống tay máy được thiết kế chuyên nghiệp hơn bằng **ros2_control**. Các khớp `arm1`, `arm2`, `ef` được đăng ký các Giao diện phần cứng (Hardware Interfaces) bao gồm `command_interface` (nhận lệnh vị trí) và `state_interface` (đọc phản hồi góc quay). Dữ liệu này được xử lý bởi bộ não `joint_trajectory_controller`.

Listing 2.3: Khai bao Hardware Interface cho ros2_control

```
<ros2_control name="GazeboSystem" type="system">
  <hardware>
    <plugin>gazebo_ros2_control/GazeboSystem</plugin>
  </hardware>
  <joint name="arm1">
    <command_interface name="position">
      <param name="min">-1.57</param>
      <param name="max">1.57</param>
    </command_interface>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
  </joint>
  </ros2_control>
```

2.2.3 Tích hợp Cảm biến LiDAR và IMU

Hai cảm biến sống còn được nhúng trực tiếp vào các thẻ `<gazebo reference="...">`. Tần số lấy mẫu (Update Rate) của LiDAR được đặt ở mức 10Hz với 360 tia quét, trong khi IMU được đặt ở 30Hz để đảm bảo phản hồi thăng bằng thời gian thực.

Listing 2.4: Khai báo cảm biến cho gazebo

```
<gazebo reference="lidar_Link">
    <sensor name="lidar_sensor" type="ray">
      <always_on>true</always_on>
      <visualize>true</visualize>
      <update_rate>10</update_rate>

      <ray>
        <scan>
          <horizontal>
            <samples>360</samples>
            <resolution>1.0</resolution>
            <min_angle>-3.14159</min_angle>
            <max_angle>3.14159</max_angle>
          </horizontal>
        </scan>
        <range>
          <min>0.15</min>
          <max>8.0</max>
          <resolution>0.015</resolution>
        </range>
        <noise>
          <type>gaussian</type>
          <mean>0.0</mean>
          <stddev>0.01</stddev>
        </noise>
      </ray>

      <plugin name="lidar_plugin" filename="
         libgazebo_ros_ray_sensor.so">
        <ros>
          <remapping>~/out:=scan</remapping>
```

```
        </ros>
        <output_type>sensor_msgs/LaserScan</output_type>
        <frame_name>lidar_Link</frame_name>
      </plugin>
    </sensor>
  </gazebo>

<gazebo reference="imu_Link"> <gravity>true</gravity>
    <sensor name="imu_sensor" type="imu">
      <always_on>true</always_on>
      <update_rate>50</update_rate>
      <visualize>true</visualize> <plugin name="imu_plugin"
         filename="libgazebo_ros_imu_sensor.so">
        <ros>
          <remapping>~/out:=imu/data</remapping>
        </ros>
        <frame_name>imu_Link</frame_name>
        <initial_orientation_as_reference>false</
           initial_orientation_as_reference>
      </plugin>
      <pose>0 0 0 0 0 0</pose>
    </sensor>
  </gazebo>
```

Cơ chế tích hợp encoder trong ROS 2 (Gazebo): Trong không gian mô phỏng, dữ liệu Encoder được giả lập với độ chính xác cao thông qua kiến trúc `ros2_control`. Các khớp động lực học được khai báo thuộc tính `<state_interface>` (đọc vị trí và vận tốc) trong file định nghĩa URDF. Dữ liệu này sau đó được thu thập, đóng gói bởi node `joint_state_broadcaster` và phát liên tục (publish) lên mạng lưới ROS thông qua topic `/joint_states` với tần số cao (50Hz - 100Hz), đảm bảo tính thời gian thực cho hệ thống.

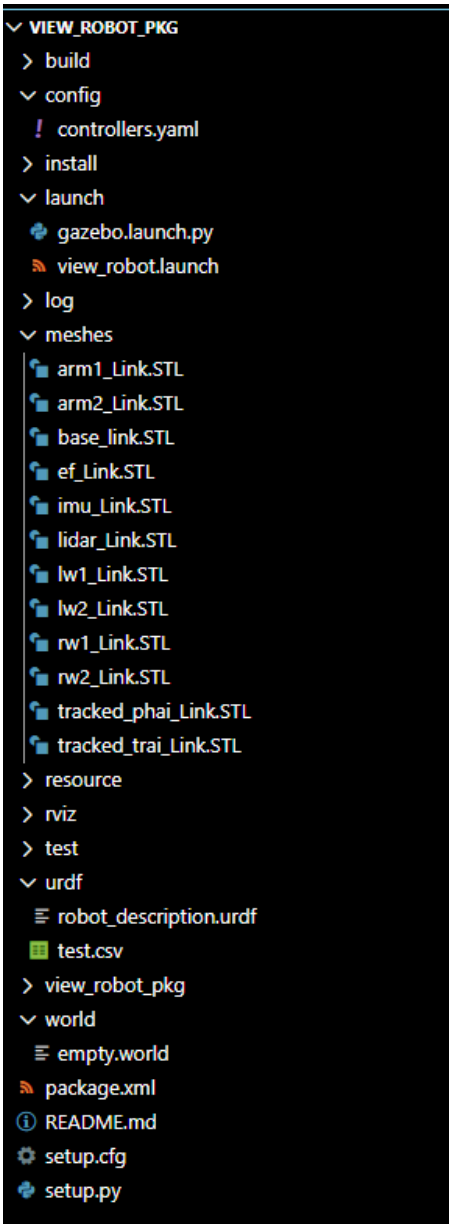
Listing 2.5: code dành cho encode trong node

```
        print("\n[ H    T H NG   T R U Y N    NG    - ENCODER
           B  N H  XE]")
        for j in ['rw1', 'rw2', 'lw1', 'lw2']:
            if j in self.joints:
                print(f"        {j:<4} : {self.joints[j
```

```
                    ]:>6.3f} rad")
```

2.2.4 Cấu trúc tổng thể của Package

Dự án được xây dựng theo kiến trúc chuẩn của một gói ROS 2 sử dụng ngôn ngữ Python (`ament_python`). Dưới đây là phân tích chi tiết chức năng của các thư mục và tệp tin cấu trúc lõi trong `view_robot_pkg`:



Hình 2.3: Mô hình được mô phỏng bởi Gazebo.

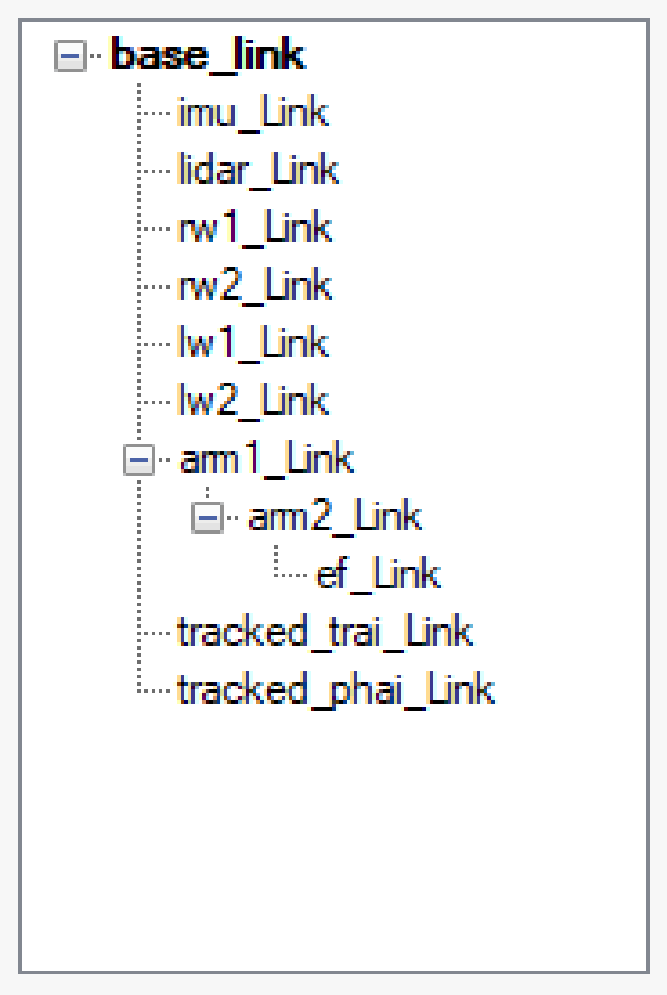
2.2.5 Cấu trúc tổng thể của package

- **Các thư mục tự sinh (build/, install/, log/):** Được tạo ra tự động khi hệ thống thực thi lệnh `colcon build`. Thư mục `install/` đóng vai trò chứa các tệp thực thi sau khi đã được đóng gói và cấp quyền môi trường.

- **config/:** Chứa các tệp tin cấu hình thông số kỹ thuật dạng `.yaml`. Cụ thể ở đây là `controllers.yaml` dùng để định nghĩa hệ thống `ros2_control`, quy định tần số hoạt động và cấu hình các bộ điều khiển tay máy và cảm biến.
- **launch/:** Thư mục đặc biệt quan trọng chứa các kịch bản khởi chạy (`.launch.py`). Thay vì phải gọi từng node riêng lẻ, các tệp launch như `gazebo.launch.py` giúp khởi động đồng thời cả hệ thống mô phỏng vật lý (Gazebo), trực quan hóa (RViz 2) và nạp tệp URDF lên không gian ROS chỉ bằng một câu lệnh.
- **meshes/:** Kho lưu trữ tài nguyên hình học 3D. Chứa toàn bộ các lưới bề mặt dưới định dạng `.STL` (như `base_link.STL`, `arm1_Link.STL`, v.v.) được trích xuất từ phần mềm CAD để khoác lên khung xương của robot.
- **urdf/:** Chứa tệp tin cốt lõi `robot_description.urdf`. Đây là tệp tin XML định nghĩa toàn bộ cây động học (Kinematic Tree), khối lượng vật lý, ma-trận quán tính và vị trí gắn các plugin cảm biến.
- **view_robot_pkg/ (Thư mục mã nguồn):** Đặc trưng của ROS 2 Python Package. Đây là nơi chứa các mã nguồn điều khiển trực tiếp (Node) viết bằng Python (tương đương với thư mục `scripts/` ở các phiên bản cũ).
- **world/:** Nơi chứa tệp cấu hình không gian vật lý `empty.world`, cung cấp ánh sáng, trọng lực và sàn phẳng cho phần mềm giả lập Gazebo.
- **Tệp tin hệ thống (package.xml, setup.py, setup.cfg):** Bộ ba tệp cấu hình bắt buộc. `package.xml` khai báo các thư viện phụ thuộc (dependencies) như `rclpy`, `ros2_control`. Tệp `setup.py` hoạt động như một trình biên dịch và đóng gói, xác định đường dẫn sao chép các tệp `launch`, `meshes`, `urdf` vào không gian thực thi.

2.2.6 Cấu trúc cây động học (Kinematic Tree) của file URDF

Mô hình robot được thiết kế trên phần mềm CAD (SolidWorks) và sử dụng plugin SW2URDF (SolidWorks to URDF Exporter) để trích xuất hệ thống tọa độ thành mã nguồn XML (`robot_description.urdf`). Dựa vào thiết lập cây thư mục trích xuất, cấu trúc phân cấp (Parent-Child) của hệ thống được định nghĩa chặt chẽ bao gồm 4 nhánh chính:



Hình 2.4: Cấu trúc cây động học của robot trên công cụ SW2URDF.

1. Liên kết gốc và Thân xe (Base Component):

- `base_link`: Đóng vai trò là trung tâm tọa độ vật lý của toàn bộ robot. Mọi bộ phận khác đều phân nhánh trực tiếp hoặc gián tiếp từ liên kết này.

2. Hệ thống truyền động (Bánh xe và Xích):

- **Bánh xe chủ động:** Bao gồm 4 liên kết `rw1_Link, rw2_Link, lw1_Link, lw2_Link` được gắn vào thân xe thông qua các khớp quay liên tục (`continuous`). Trục quay được chỉ định là trục Z (`<axis xyz="0 0 1"/>`) để mô phỏng sự lăn của bánh.

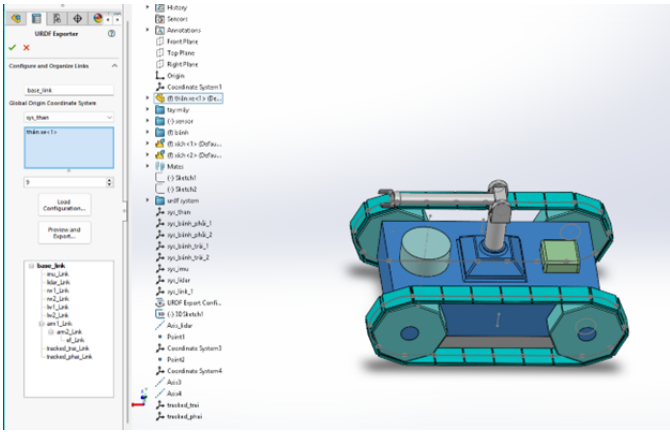
- **Dải xích ảo:** Gồm `tracked_trai_Link` và `tracked_phai_Link` được gắn cố định (`fixed`) vào thân. Khối lượng của hai dải xích này được cố tình thiết lập bằng 0 (`<mass value="0"/>`) nhằm mục đích thuần hiển thị (visual), giúp động cơ Gazebo không bị quá tải khi tính toán ma sát phức tạp của mắt xích.

3. Hệ thống cơ cấu chấp hành (Tay máy 2-DOF): Hình thành một chuỗi động học mở (Open Kinematic Chain) phân cấp nối tiếp nhau:

- `arm1_Link` (Khớp vai): Nối với thân xe qua khớp `revolute`, bị giới hạn góc quay trong khoảng an toàn ± 1.57 rad để tránh va đập vào gầm xe.
- `arm2_Link` (Khớp khuỷu): Phân nhánh từ `arm1_Link` qua khớp `revolute`, cho phép gập duỗi từ 0 đến 3.14 rad.
- `ef_Link` (Khớp tác động cuối/Nòng súng): Phân nhánh từ `arm2_Link`.

4. Hệ thống cảm biến (Sensors):

- `imu_Link` và `lidar_Link` đều được khai báo bằng khớp `fixed` trực tiếp vào `base_link`. Việc gắn cố định cứng này đảm bảo hệ quy chiếu dữ liệu quét (LaserScan) và dữ liệu quán tính (IMU) luôn đồng bộ tuyệt đối với tọa độ của thân xe, là cơ sở toán học bắt buộc cho các thuật toán nội suy Odometry và xây dựng bản đồ (SLAM).



Hình 2.5: Hình ảnh hoàn chỉnh cây động học của robot trên công cụ SW2URDF.

2.2.7 Giải phẫu cấu trúc XML của tệp URDF trích xuất

Khi được trích xuất từ phần mềm SolidWorks, mỗi thành phần của robot (Link) và mối liên kết (Joint) được cấu trúc hóa theo các thẻ XML tiêu chuẩn. Việc hiểu rõ cấu trúc này là bắt buộc để có thể tinh chỉnh các thông số vật lý và cảm biến mà phần mềm CAD chưa xử lý hoàn thiện.

1. Cấu trúc thẻ <link> (Thành phần vật lý): Mỗi Link trong file URDF của chiến thần Orange gồm ba khối dữ liệu chính:

- `<visual>`: Định nghĩa diện mạo của robot. Thẻ này chứa đường dẫn tới file lưới `.STL` trong thư mục `meshes/`. Đây là phần mắt chúng ta thấy trong Gazebo và RViz.

- `<collision>`: Định nghĩa vùng va chạm vật lý. Để tối ưu hóa hiệu năng, vùng va chạm thường được bao bọc đơn giản hơn so với mô hình hiển thị nhưng vẫn phải đảm bảo bao quát đủ thể tích thực của linh kiện.

- `<inertial>`: Trái tim vật lý của robot. Bao gồm trọng tâm (`origin`), khối lượng (`mass`) và ma-trận quán tính (`inertia`). *Lưu ý kỹ thuật: Ma-trận quán tính trích xuất từ CAD thường rất phức tạp; trong đồ án này, các giá trị quán tính của bánh xe đã được hiệu chỉnh về dạng đường chéo (**`ixx, iyy, izz`**) để tăng độ ổn định cho engine vật lý ODE.*

2. Cấu trúc thẻ <joint> (Hệ thần kinh vận động): Thẻ Joint thiết lập mối quan hệ giữa hai Link (Parent và Child). Trong URDF của robot, có hai loại khớp quan trọng:

- **Khớp xoay giới hạn (revolute):** Áp dụng cho `arm1` và `arm2`. Cấu trúc này bắt buộc phải có thẻ `<limit>` để quy định góc quay tối đa và tối thiểu (ví dụ: khớp khuỷu `arm2` giới hạn từ 0 đến 3.14 rad).

- **Khớp xoay liên tục (continuous):** Áp dụng cho 4 bánh xe (`rw1, lw1...`), cho phép bánh xe quay vô hạn quanh trục dẫn động.

3. Thẻ <transmission> và <gazebo>: Đây là các thẻ "mở rộng"được bổ sung sau quá trình export:

- `<transmission>`: Đóng vai trò là cầu nối giữa bộ phận cơ khí và động cơ ảo. Nó khai báo loại phần cứng điều khiển (`PositionJointInterface`) để `ros2_control` có thể "nhận lệnh"và truyền lực vào khớp.

- <gazebo>: Chứa các thuộc tính riêng cho môi trường mô phỏng như màu sắc hiển thị (Gazebo/Orange, Gazebo/Black) và các plugin cảm biến LiDAR, IMU.

Listing 2.6: Code cơ bản của 1 link trong urdf

```
<link
  name="base_link">
  <inertial>
    <origin
      xyz="-3.4184E-05 -2.1627E-13 0.020987"
      rpy="0 0 0" />
    <mass
      value="3" />
    <inertia
      ixx="0.5"
      ixy="0"
      ixz="0"
      iyy="0.08"
      iyz="0"
      izz="0.05" />
  </inertial>
  <visual>
    <origin
      xyz="0 0 0"
      rpy="0 0 0" />
    <geometry>
      <mesh
        filename="package://view_robot_pkg/meshes/
           base_link.STL" />
    </geometry>
    <material
      name="">
      <color
        rgba="1 0.94902 0.89804 1" />
    </material>
  </visual>
  <collision>
    <origin
```

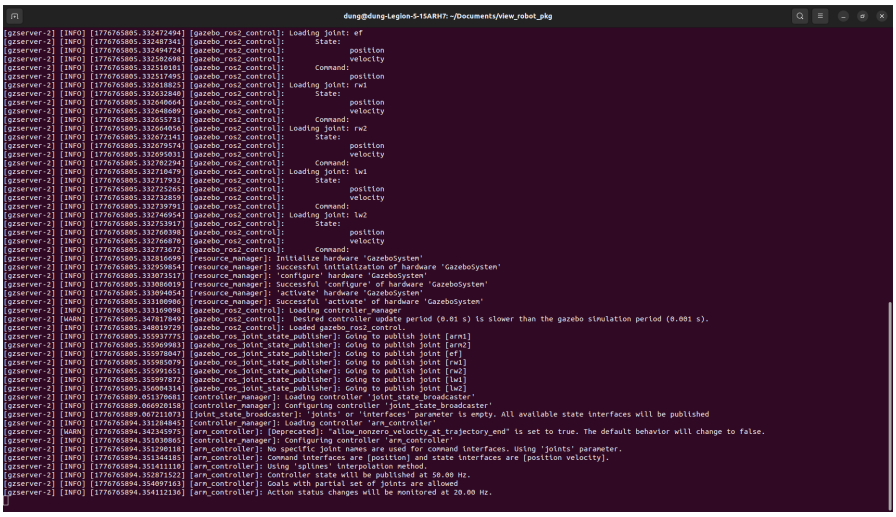
```
        xyz="0 0 0"
        rpy="0 0 0" />
      <geometry>
        <mesh
          filename="package://view_robot_pkg/meshes/
             base_link.STL" />
      </geometry>
    </collision>
  </link>
```

Chương 3

Kết quả thu được

3.1 Khởi chạy chương trình

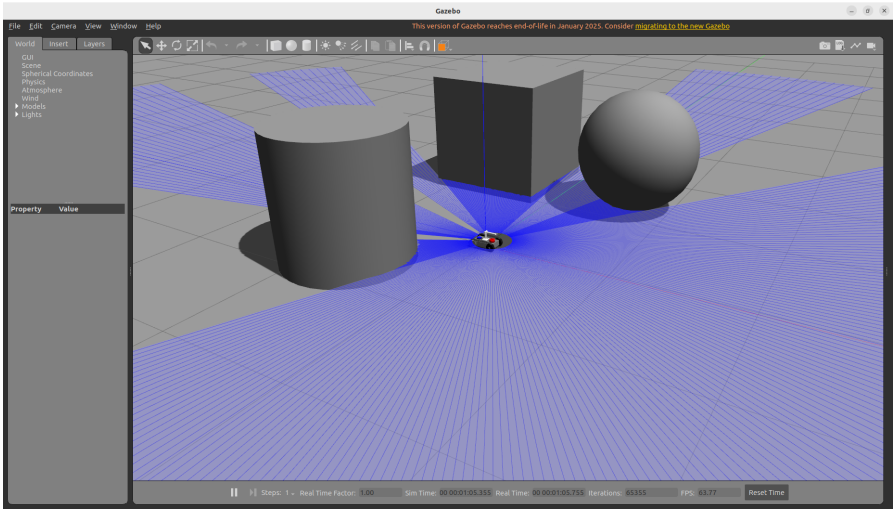
Tiến hành khởi chạy các file launch và quan sát các node hoạt động.



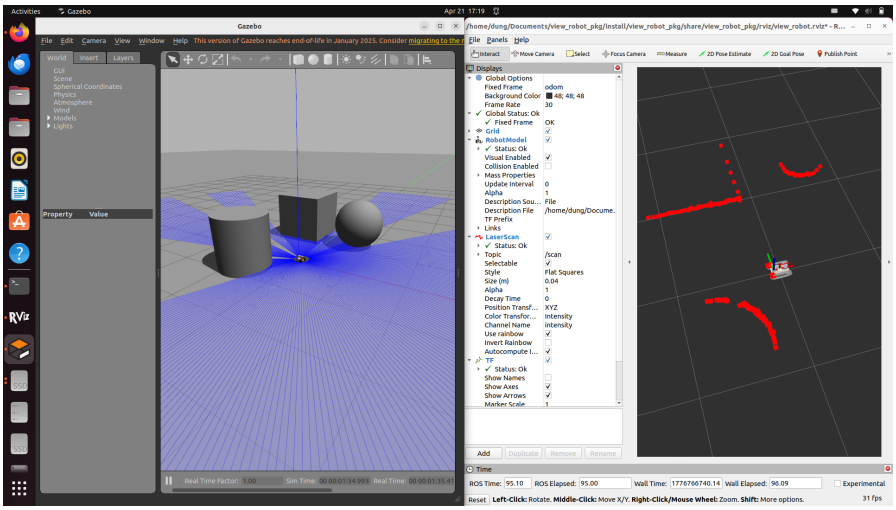
Hình 3.1: Khởi chạy chương trình.

3.2 Hiển thị trong mô phỏng

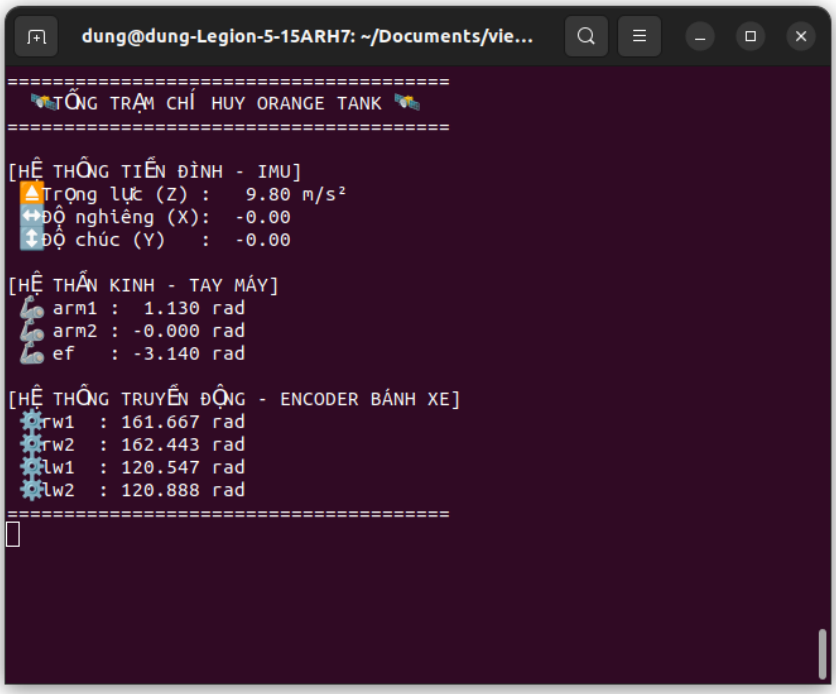
Robot được load thành công vào Gazebo và RVIZ với đầy đủ các trục toạ độ và cảm biến hoạt động ổn định.



Hình 3.2: Robot hiển thị trong Gazebo.



Hình 3.3: Robot hiển thị trong RVIZ.



Hình 3.4: Các thông số của của cảm biến imu và encoder.

Chương 4

Kết luận

4.1 Đã thực hiện

- Thiết kế thành công mô hình 3D cho robot bánh xích tích hợp tay máy 2 bậc tự do.
- Cấu hình thành công môi trường mô phỏng trên Gazebo và trực quan hóa qua RVIZ.
- Điều khiển được robot di chuyển các bánh và tay máy thông qua Teleop.
- Thu thập và hiển thị được kết quả dữ liệu từ các cảm biến.

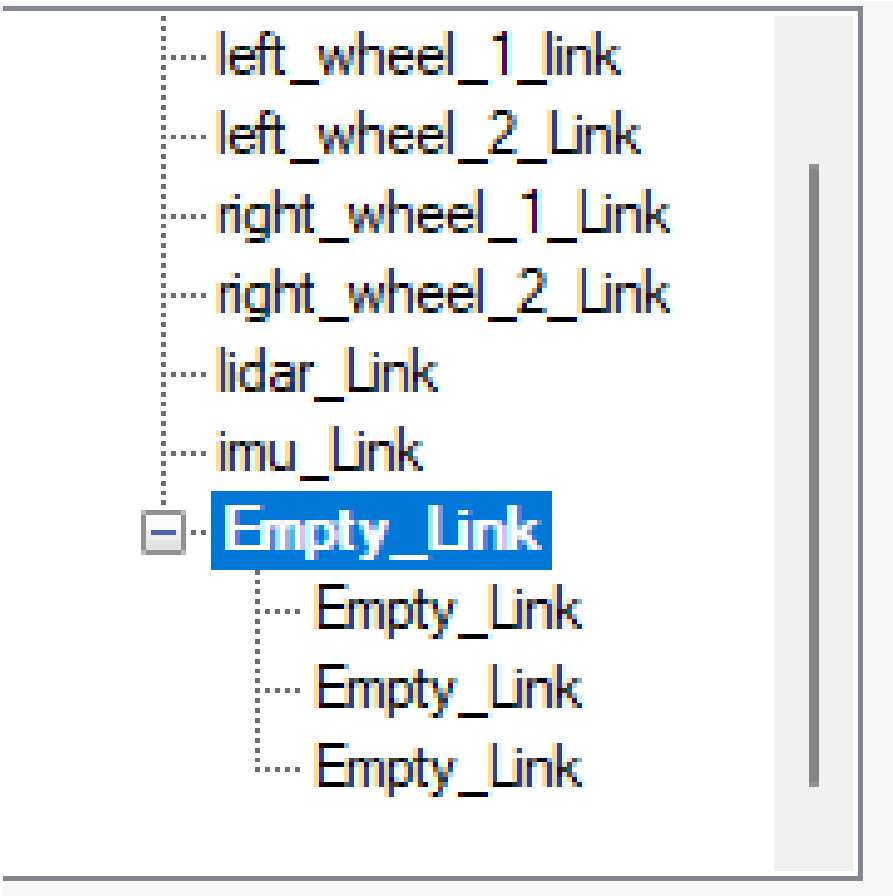
4.2 Công việc cần hoàn thiện

- Điều khiển di chuyển bánh xe của robot chính xác hơn trong các quỹ đạo hẹp.
- Cải tiến cách di chuyển của các khớp tay máy và hệ truyền động để robot không bị khựng hay giật cục khi tăng tốc.

Các lỗi phát sinh trong quá trình làm và cách sửa

4.3 Lỗi liên quan đến việc xuất URDF

4.3.1 Lỗi quan hệ cấu trúc cây - tay máy



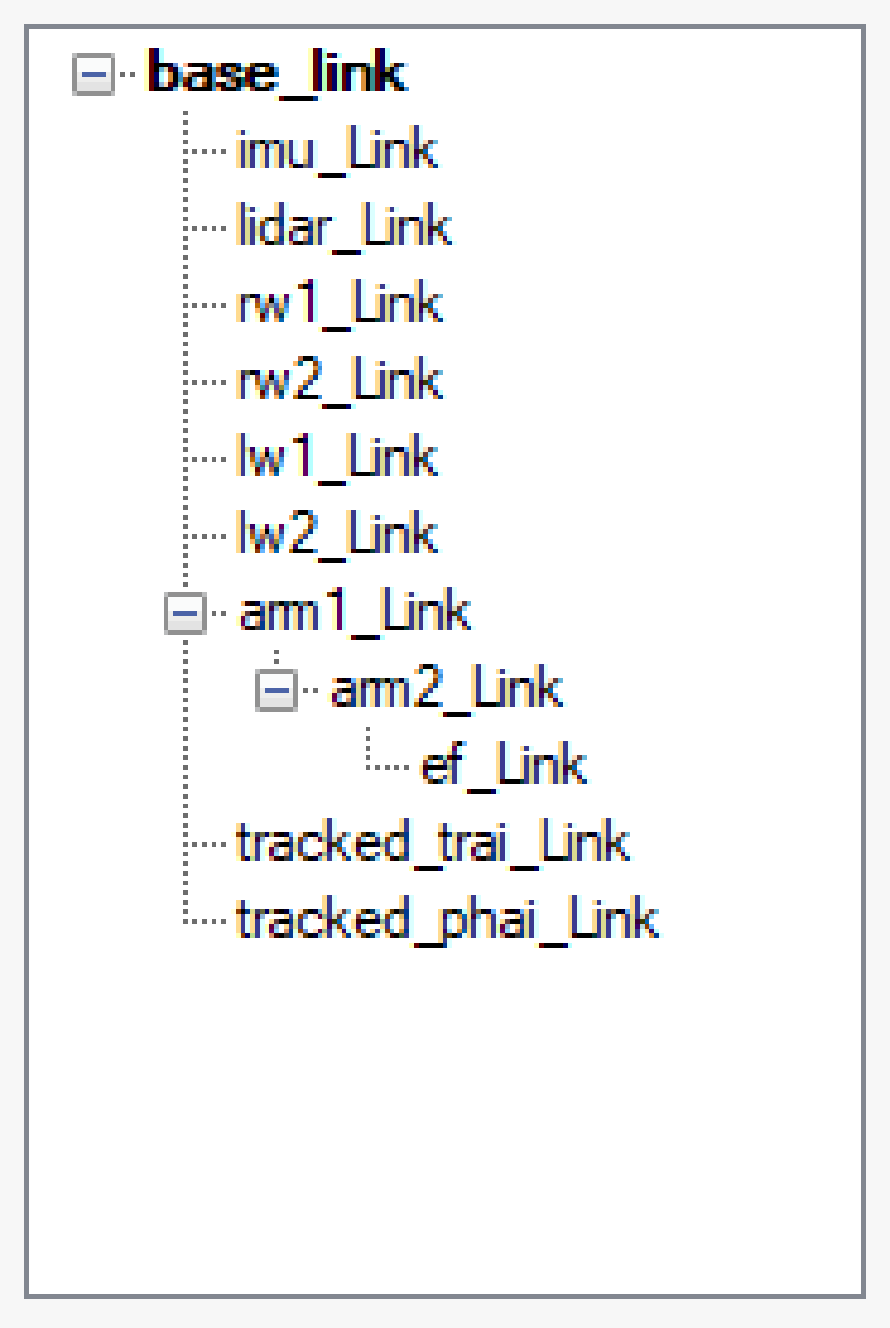
Hình 4.1: cây bị lỗi.

Lỗi:

- Đây vốn là vị trí của 3 khớp tay máy Arm1, Arm2, Ef.
- Việc để thiết kế cây như này sẽ khiến cho các khớp tay máy tưởng có liên kết với nhau sẽ ko có mối liên kết và không thể nào điều khiển được.

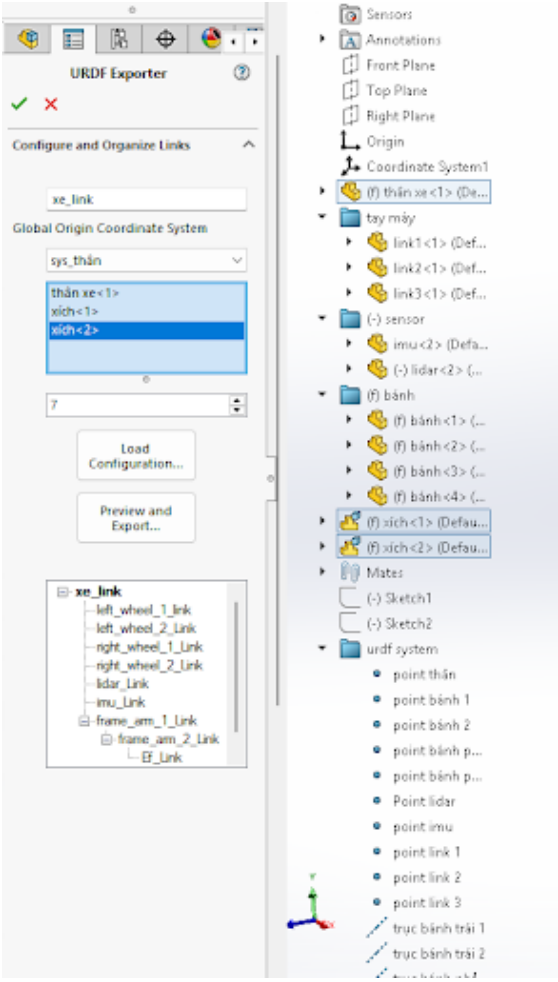
Phương án sửa chữa:

- Điều chỉnh lại các mỗi quan hệ giữa Arm1, Arm2, Ef.
- Gán mối quan hệ Parent - Child cho các link tay máy để đảm báo chúng có liên kết với nhau để có thể điều khiển về sau.



Hình 4.2: Hình ảnh cây quan hệ chuẩn sau khi đổi.

4.3.2 Lỗi quan hệ cấu trúc cây - xích



Hình 4.3: Link components bị thừa.

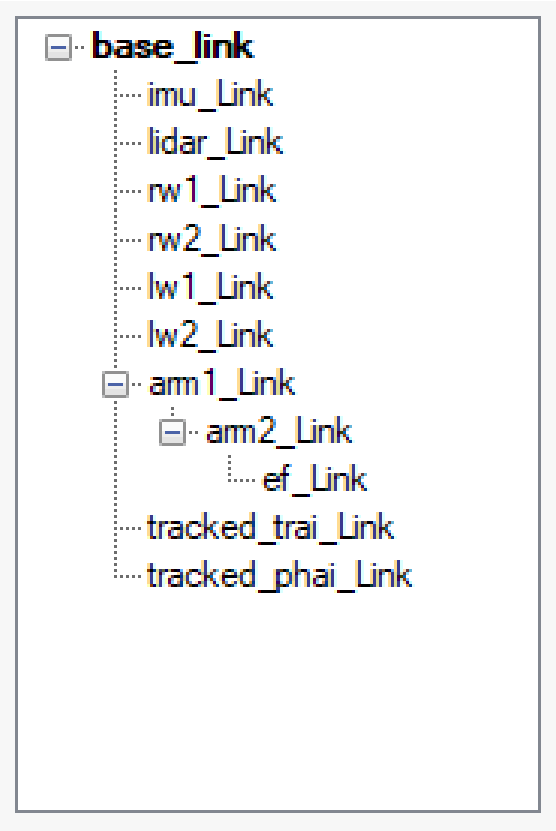
Lỗi:

- Việc gán bánh xích vào link components của base link sẽ khi cho mesh của vật thể bị lỗi và gây khó khăn cho việc tính toán quán tính.
- Điều này sẽ khiến cho bánh xích mất đi tính chất vật lý cũng như khiến cho trục của xe có xu hướng chìm trực tiếp dưới $z = 0$.

Phương án sửa chữa:

- Tạo 2 link riêng cho 2 bánh xe.

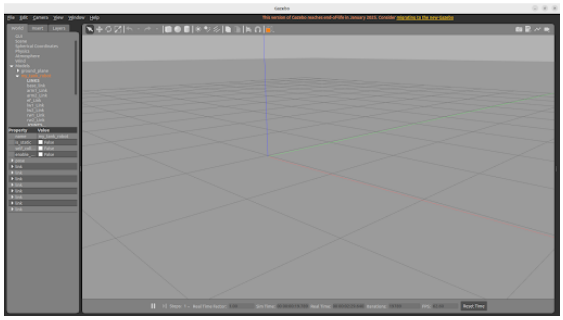
- Gán mối quan hệ Parent - Child cho các bánh xích để đảm báo chúng có liên kết với base link để có thể điều khiển về sau.



Hình 4.4: Hình ảnh cây quan hệ chuẩn sau khi đổi.

4.4 Lỗi liên quan đến mô phỏng trên gazebo

4.4.1 Lỗi nổ vật lý của agv cũng như không hiên agv khi nhét vô mô phỏng vật lý gazebo



Hình 4.5: Hình ảnh cây quan hệ chuẩn sau khi đổi.

Nguyên nhân:

- Do trong file Urdf đã thiếu đi các thông số liên quan đến ma sát giữa mặt đất và bánh xe nên khiến cho xe bị rơi tự do mất kiểm xoát .
- Ngoài ra lý do mà xe không hiện là gazebo đã không thể nhúng trực tiếp vô file mesh (hay file STL cấu hình bề mặt ngoài của xe).

Listing 4.1: code của bánh xe khi trước khi thêm các yếu tố vật lý cần thiết

```
  <link
  name="rw1_Link">
  <inertial>
    <origin
      xyz="-1.1102E-16 0 0"
      rpy="0 0 0" />
    <mass
      value="0.3" />
    <inertia
      ixx="3E-04"
      ixy="0"
      ixz="0"
      iyy="3E-04"
```

```
        iyz="0"
        izz="3E-04" />
    </inertial>
    <visual>
      <origin
        xyz="0 0 0"
        rpy="0 0 0" />
      <geometry>
        <mesh
          filename="package://view_robot_pkg/meshes/rw1_Link
             .STL" />
      </geometry>
      <material
        name="">
        <color
          rgba="1 1 1 1" />
      </material>
    </visual>
    <collision>
      <origin
        xyz="0 0 0"
        rpy="0 0 0" />
      <geometry>
        <cylinder radius="0.033" length="0.02" />
      </geometry>
    </collision>
  </link>
  <joint
    name="rw1"
    type="continuous">
    <origin
      xyz="-0.075 -0.06 0.02"
      rpy="-1.5708 0 0" />
    <parent
      link="base_link" />
    <child
      link="rw1_Link" />
    <axis
```

```
      xyz="0 0 1" />
  </joint>
```

Listing 4.2: Terminal báo không tìm được mesh

```
    Err] [MeshShape.cc:64] Failed to find mesh file [model
       ://view_robot_pkg/meshes/base_link.STL]

    [gzserver-1] [Wrn] [FuelModelDatabase.cc:313] URI not
       supported by Fuel [model://view_robot_pkg/meshes/
       imu_Link.STL]

    [gzserver-1] [Wrn] [SystemPaths.cc:459] File or path
       does not exist [""]
```

Phương án sửa chữa:

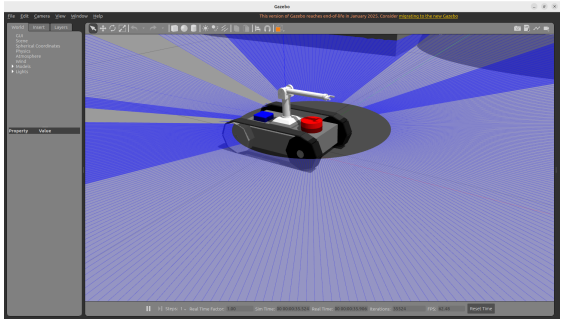
- để xe không bị rơi thì phải gắn thêm các yếu tố vật lý cho từng bánh xe cũng như vẽ collision thành các hình khối đơn giản để dễ dàng cho việc tính toán vật lý trên gazebo.
- Điều chỉnh lại code launch gazebo để chúng có thể chỉ trực tiếp vô file mesh chứ không phải thông qua file URDF.

Listing 4.3: điều chỉnh collision cũng như là thêm thông số vật lý

```
<geometry>
    <cylinder radius="0.033" length="0.02" />
</geometry>
    ...
  <gazebo reference="rw1_link">
    <mu1>0.8</mu1>
    <mu2>0.8</mu2>
    <kp>1000000.0</kp>
    <kd>100.0</kd>
    <material>Gazebo/Black</material>
  </gazebo>
```

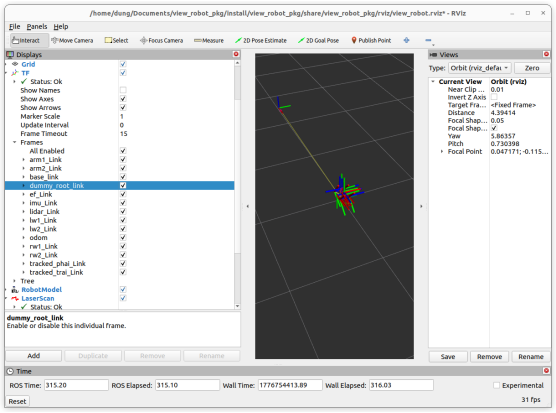
Code chỉ đường dẫn trực tiếp vô file mesh trong file Launch

```
pkg_parent = os.path.dirname(pkg_share)
set_gazebo_model_path = SetEnvironmentVariable(
    name='GAZEBO_MODEL_PATH',
    value=pkg_parent
)
```



Hình 4.6: Xe Agv hiện sau khi sửa các lỗi.

4.4.2 Lỗi lidar không nhận diện được vật thể



Hình 4.7: Xe Agv hiện sau khi sửa các lỗi.

Nguyên nhân:

- Do code gốc trong file urdf đã khai báo sai tên của Link Lidar dẫn đến việc khi chạy mô phỏng, Rviz tuy vận nhận có lidar nhưng nó là lidar ảo mà không hề có tín hiệu

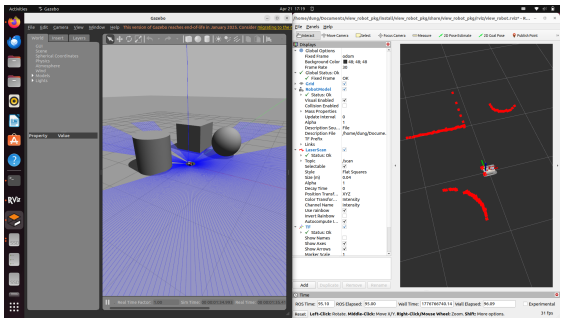
Listing 4.4: Code lidar gốc

```
<<gazebo reference="lidar">
    <sensor name="lidar_sensor" type="ray">
      <always_on>true</always_on>
      <visualize>true</visualize>
      <update_rate>10</update_rate>
      <ray>
        <scan>
          <horizontal>
            <samples>360</samples>
            <resolution>1.0</resolution>
            <min_angle>-3.14159</min_angle>
            <max_angle>3.14159</max_angle>
          </horizontal>
        </scan>
        <range>
          <min>0.15</min>
          <max>8.0</max>
          <resolution>0.015</resolution>
        </range>
        <noise>
          <type>gaussian</type>
          <mean>0.0</mean>
          <stddev>0.01</stddev>
        </noise>
      </ray>
      <plugin name="lidar_plugin" filename="
         libgazebo_ros_ray_sensor.so">
        <ros>
          <remapping>~/out:=scan</remapping>
        </ros>
        <output_type>sensor_msgs/LaserScan</output_type>
        <frame_name>lidar</frame_name>
      </plugin>
    </sensor>
  </gazebo>
```

Phương án sửa chữa:

- điều chỉnh lại code URDF cho phù hợp với cây quan hệ các thành phần.

4.4.3 Lỗi lidar không nhận diện được vật thể



Hình 4.8: Xe Agv hiện sau khi sửa các lỗi.

4.5 Thành phẩm sau cùng cũng như link github sản phẩm

- **Mã nguồn trên GitHub:** Click để xem GitHub Repository
- **vid thành phẩm (Google Drive):** Truy cập thư mục Google Drive